Loop Engineering: A Conceptual Framework for Reliable Execution in AI Agent Systems

Author: Mike, AI Tool Insider
Date: June 17, 2026
Paper type: Conceptual framework paper with design-science synthesis
Citation style: APA 7

Abstract

AI agents are increasingly expected to execute multi-step tasks through planning, tool use, memory, verification, and revision. However, many agent failures arise not from model incapacity alone but from weak orchestration: unbounded loops, poor state tracking, unclear termination, and insufficient verification. This paper proposes **loop engineering** as a conceptual framework for reliable execution in AI agent systems. The framework treats the repeated observe–plan–act–evaluate cycle as the primary engineering unit of agentic behavior. It synthesizes prior work in ReAct, Reflexion, AutoGen, LangGraph, state machines, workflow orchestration, control theory, reinforcement learning, and planner–executor architectures.

Unlike systems that treat looping as an implicit prompting pattern, loop engineering proposes explicit mechanisms for inspection, bounded execution, and governance. The paper defines core concepts including executable AI agents, state models, reflective state, governance loops, and loop traces. It proposes a six-component framework: goal representation, state model, action executor, observation collector, evaluator, and controller. The paper also introduces a design-science-inspired methodology, an evidence map, a qualitative evaluation rubric, and a discussion of controller policies such as confidence thresholds, cost budgets, risk budgets, progress scores, entropy thresholds, and human escalation.

The paper further discusses limitations and counterarguments. Loop-based execution is not universally appropriate; event-driven, reactive, graph-based, and one-shot workflows may be better in some contexts. The paper concludes that loop engineering is not a universal theory of AI agents, but it is a useful framework for AI agents that must complete complex tasks under uncertainty.

Keywords: AI agents, loop engineering, agentic AI, ReAct, Reflexion, AutoGen, LangGraph, workflow orchestration, agent governance, tool use

---

1. Introduction

AI agents are moving beyond single-turn text generation toward systems that can plan, call tools, inspect outputs, revise decisions, and complete workflows over time. This shift changes the engineering

problem. The central challenge is no longer only how to produce a good answer in one model call, but how to design a system that can operate safely and effectively across many steps.

This paper introduces **loop engineering** as a conceptual framework for reliable execution in AI agent systems. Loop engineering refers to the deliberate design of repeated execution cycles in which an agent observes its current state, selects an action, executes that action, evaluates the result, and decides whether to continue, revise, ask for help, or stop. The model is not the entire agent; it is one component inside a broader control system.

The contribution of this paper is not that it invents the loop as a computational idea. Loops, feedback, state machines, planner–executor systems, workflow orchestration, and control-theoretic reasoning predate modern AI agents. Rather, the contribution is to synthesize these ideas into a framework for designing and evaluating AI agents where the **loop itself becomes the engineering unit**.

The paper makes four claims:

1. Many practical AI agent failures are orchestration failures rather than only model failures.
2. Loop engineering provides a useful abstraction for organizing planning, tool use, memory, verification, and termination.
3. Reliable loop-engineered agents require explicit state, bounded execution, evaluators, and governance mechanisms.
4. Loop-based execution is not universally appropriate; event-driven, reactive, graph-based, and one-shot workflows may be better in some contexts.

This framing is intentionally qualified. The paper does not claim that loop engineering is a necessary foundation for all AI agent systems. Instead, it argues that loop engineering may provide a useful framework for a class of AI agent systems that must execute multi-step tasks under uncertainty.

---

2. Literature Review and Conceptual Positioning

2.1 Related Work

The idea of agents that perceive, reason, and act has roots in classical AI and cybernetics. Russell and Norvig (2021) define agents as entities that perceive their environment and take actions. Cybernetics and control theory emphasize feedback as a mechanism for regulating systems over time (Wiener, 1948). State machines and workflow systems provide formal ways to represent transitions between states, while reinforcement learning studies agents that learn policies through interaction with environments (Sutton & Barto, 2018).

Modern AI agent systems adapt these older ideas to language models. ReAct demonstrates that language models can improve task performance by interleaving reasoning traces with actions such as searching or tool use (Yao et al., 2023). Reflexion adds self-reflection and memory so agents can learn from failed attempts and improve future behavior (Shinn et al., 2023). AutoGen shows how multiple

agents can collaborate through conversation, with specialized roles such as planner, coder, reviewer, or user proxy (Wu et al., 2023). LangGraph represents agent workflows as state graphs, making transitions between planning, execution, review, and termination explicit (LangChain, 2024).

These systems do not use identical architectures, but they share a common pattern: an agent takes an action, observes the result, updates its understanding, and chooses the next step. Loop engineering reframes this shared pattern as an explicit design object.

2.2 Conceptual Gap

The central conceptual gap is not the existence of loops in AI systems. Loops are already present in control theory, reinforcement learning, state machines, workflow engines, and planner–executor architectures. The gap is that many modern AI agent systems treat looping as an implicit byproduct of prompting rather than as an explicit engineering layer.

For example, a ReAct-style prompt may ask a model to “think, act, observe, and repeat,” but the system may not formally define when repetition should stop, how state should be updated, how tool outputs should be validated, or how failures should be logged. Similarly, a multi-agent conversation may continue until a model decides to stop, but the conversation itself may lack explicit progress metrics or rollback procedures.

Loop engineering proposes explicit mechanisms for inspection, bounded execution, and governance. It does not replace ReAct, Reflexion, AutoGen, or LangGraph. Instead, it provides a cross-cutting lens for analyzing and improving them.

2.3 Relationship to Existing Concepts

Existing concept	Relationship to loop engineering
Control theory	Provides the feedback principle: observe output, compare to goal, adjust action
State machines	Provide formal state transitions and termination conditions
Reinforcement learning	Provides action–observation–reward cycles
OODA loop	Provides a decision cycle: observe, orient, decide, act
Planner–executor architectures	Separate planning from execution and verification
Workflow orchestration	Provides bounded task flows, retries, and human approval
ReAct	Provides reasoning–action interleaving for language models
Reflexion	Provides memory and self-reflection after failures
AutoGen	Provides multi-agent conversational coordination
LangGraph	Provides state-graph representation of agent workflows

Loop engineering is therefore not a replacement for these ideas. Its contribution is to organize them around the practical problem of **reliable execution in AI agent systems**.

2.4 Evidence Map and Its Limits

The evidence for loop engineering comes from several overlapping areas. These domains support portions of the framework but do not directly validate loop engineering as a unified construct.

Evidence area	What it supports	Limitation
Control theory	Feedback improves regulation of dynamic systems	Does not directly study LLM agents
State machines	Explicit states reduce ambiguity	May be too rigid for open-ended language-model reasoning
Workflow orchestration	Bounded workflows improve operational reliability	Often assumes predefined tasks rather than emergent plans
ReAct	Interleaving reasoning and action can improve task solving	Does not fully solve termination, rollback, or governance
Reflexion	Reflection and memory can reduce repeated failures	Reflection quality depends on evaluation signals
AutoGen	Role specialization can improve complex task execution	Multi-agent conversation can become unbounded
LangGraph	State graphs make agent workflows inspectable	Graph structure must still be designed carefully
Software engineering	Verification, testing, and rollback reduce execution risk	Does not address model-specific uncertainty

This evidence map supports a narrower claim: when AI agents must execute multi-step tasks under uncertainty, explicit loop design can improve reliability, transparency, and controllability. It does not prove that loop engineering is universally superior.

2.5 Summary of Conceptual Positioning

Loop engineering is best understood as a **synthesis framework** rather than a new computational primitive. It draws from established ideas in control, state management, workflow orchestration, reinforcement learning, and AI agent design. Its novelty lies in applying those ideas to the practical engineering of language-model-based agents whose execution depends on repeated action, observation, evaluation, and control.

This positioning avoids the claim that loop engineering replaces existing frameworks. Instead, it provides a vocabulary and design structure for making agentic execution more reliable.

---

3. Methodology

This paper uses a **design-science-inspired conceptual synthesis methodology**. Design science is appropriate because the paper develops an artifact-oriented framework for building and evaluating AI agent systems rather than testing a natural phenomenon through controlled experiment (Hevner et al., 2004).

The methodology has three parts.

3.1 Literature Synthesis

The paper reviews major concepts and systems relevant to AI agent execution: control theory, state machines, workflow orchestration, ReAct, Reflexion, AutoGen, and LangGraph. These sources are used to identify recurring components of agent execution.

The synthesis focuses on five recurring mechanisms:

1. **Feedback:** the agent observes the result of prior actions.
2. **State:** the agent maintains a record of task progress.
3. **Action:** the agent performs operations through tools or other agents.
4. **Evaluation:** the agent assesses whether progress has occurred.
5. **Control:** the system decides whether to continue, revise, stop, or escalate.

3.2 Research Questions

This paper is guided by three research questions:

Research question	Purpose
RQ1: What components are necessary for loop-engineered AI agents?	Defines the framework
RQ2: How does loop engineering differ from informal agent prompting?	Clarifies the conceptual contribution
RQ3: How could loop-engineered agents be evaluated?	Provides a path toward empirical validation

These questions are conceptual rather than empirical. They support framework construction rather than statistical hypothesis testing.

3.3 Framework Construction

The paper defines a loop-engineering framework with six core components:

1. Goal representation
2. State model
3. Action executor
4. Observation collector
5. Evaluator
6. Controller

These components are then organized into a repeatable execution cycle.

3.4 Qualitative Evaluation Design

Rather than presenting invented numerical results, this paper uses a qualitative evaluation framework. This avoids the credibility problem of unsupported percentages while still making the framework testable.

The evaluation dimensions are:

Dimension	Evaluation question
Goal fidelity	Does the agent remain aligned with the original task?
State continuity	Does the agent preserve requirements, outputs, and errors across iterations?
Recovery	Can the agent recover after tool errors, ambiguous outputs, or failed plans?
Termination	Does the agent stop when the goal is achieved or when progress stalls?
Governance	Does the agent avoid unsafe, unauthorized, or high-cost actions?
Traceability	Are actions, observations, decisions, and revisions logged?

This qualitative design is suitable for a conceptual paper. Future empirical work could replace or supplement it with controlled benchmarks, case studies, or reproducible simulations.

3.5 Methodological Limitations

This methodology does not claim to prove causal effectiveness. It provides a structured framework and evaluation language. The framework should be treated as a design proposal that requires empirical testing.

---

4. Proposed Framework: Loop Engineering for AI Agent Execution

4.1 Definition

Loop engineering is the deliberate design, monitoring, and governance of repeated AI agent execution cycles. A loop-engineered agent does not merely generate a response; it continuously updates its state through action, observation, evaluation, and control.

A loop-engineered agent can be represented as:

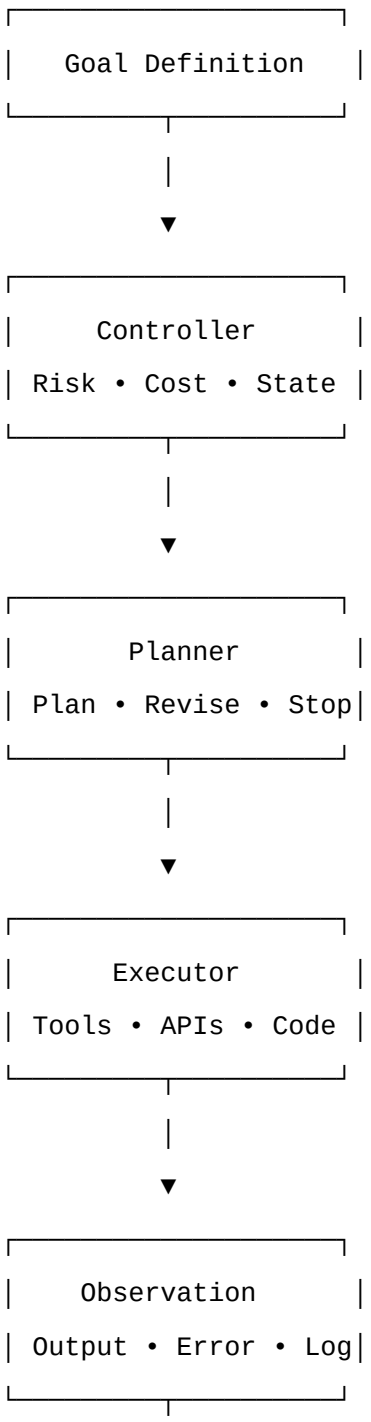
```
Goal → Controller → Planner → Executor → Observation → State Update → Evaluator → Continue / Stop
```

The controller may then choose to:

- Continue the current plan.
- Revise the plan.
- Retry the previous action.

- Roll back a change.
- Ask for human approval.
- Terminate successfully.
- Terminate with failure.

4.2 Figure 1: Loop-Engineering Architecture



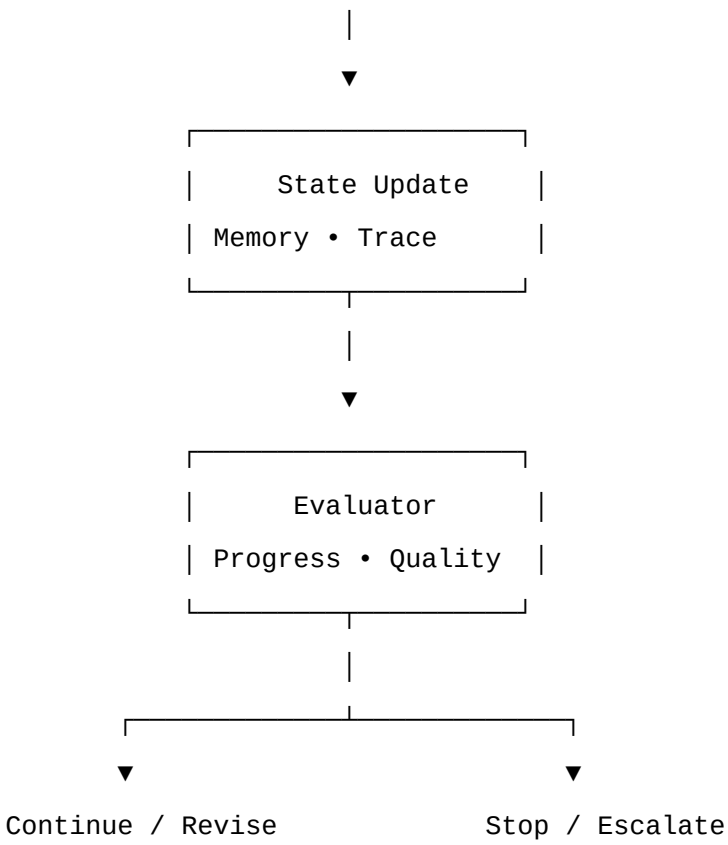


Figure 1. Loop-engineering architecture for executable AI agents.

4.3 Core Components

4.3.1 Goal Representation

The goal representation defines what the agent is trying to accomplish. A weak goal representation may be vague, such as “write a paper about AI agents.” A stronger representation includes task boundaries, success criteria, constraints, and stop conditions.

A structured goal representation may include:

```
Task: Write a conceptual paper on AI agents that execute through loop engineering.

Success criteria:
- Clear thesis
- Literature review
- Conceptual framework
- Evaluation discussion
- Limitations
```

```
- APA-style references

Constraints:
- Avoid hype
- Do not claim loops are entirely new
- Include counterarguments

Stop conditions:
- Paper draft is complete
- Maximum revision iterations reached
- Human approval received
```

Goal representation is

4.4.3 Risk Budget

A **risk budget** defines what kinds of actions the agent may take without approval. Risk may be based on the action type, target system, irreversibility, data sensitivity, financial impact, or external visibility.

Example risk categories:

Risk level	Example action	Controller behavior
Low	Read a public webpage	Allow automatically
Medium	Write a local draft file	Allow with logging
High	Delete files, send emails, or modify shared documents	Require approval
Critical	Modify production systems, spend money, or expose private data	Require explicit human authorization

Risk budgets are especially important because agents can act in environments where mistakes have consequences. A loop-engineered agent should not only ask, “What should I do next?” but also, “Am I allowed to do this?”

The risk budget should be domain-specific. For example, a research assistant may be allowed to read public sources and draft summaries without approval, but it should not submit manuscripts, send emails, or make claims about private data without human review. A coding agent may be allowed to edit local test files, but it may need approval before modifying production infrastructure or deleting generated artifacts.

4.4.4 Progress Score

A **progress score** estimates how much closer the agent is to the goal after each iteration. It can be qualitative, rubric-based, or numeric.

A simple conceptual version may be expressed as:

```
Progress score = goal coverage + correctness + completeness - drift - repeated errors
```

A qualitative version may use labels:

Progress score	Meaning	Controller response
High	The latest action clearly advanced the task	Continue
Medium	Some progress, but uncertainty remains	Continue with additional verification
Low	Little or no progress	Revise plan
Negative	The action moved away from the goal	Roll back or escalate

Progress scoring helps prevent loops from continuing merely because the agent is active. Activity is not the same as progress.

Progress scoring is especially useful in open-ended tasks where there is no single deterministic success condition. For example, in writing a research paper, progress may include completing the literature review, adding citations, addressing counterarguments, and improving the methodology section. In a coding task, progress may include passing additional tests, reducing error count, or improving code coverage.

4.4.5 Entropy Threshold

An **entropy threshold** detects uncertainty, instability, or disagreement in the agent's state. In this context, entropy does not need to be mathematically precise. It can refer to signs that the agent's understanding is unstable.

Examples of high entropy include:

- The evaluator changes its judgment across repeated iterations.
- Tool outputs conflict with one another.
- The agent alternates between incompatible plans.
- User requirements are ambiguous or contradictory.
- The model repeatedly produces different answers to the same subtask.
- Retrieved sources disagree on a key claim.

When entropy exceeds a threshold, the controller should stop blindly iterating and switch to clarification, evidence gathering, or human escalation.

This policy is important because many agent failures occur when the system keeps acting despite uncertainty. A high-entropy state should trigger a different behavior: ask a question, search for better evidence, consult another agent, or pause for human judgment.

4.4.6 Human Escalation

A **human escalation policy** defines when the agent must involve a person. Escalation may be triggered by high risk, low confidence, high entropy, budget exhaustion, user-requested approval, or repeated failure.

Examples:

```
Escalate if:
- Proposed action is irreversible
- Confidence is below threshold after two verification attempts
- Progress score remains low after plan revision
- The task requires ethical, legal, medical, financial, or safety judgment
- The agent detects conflicting user instructions
- The cost budget is nearly exhausted
- The agent is about to modify a production system
```

Human escalation should be treated as a first-class loop decision, not as a failure state. It is a governance mechanism that allows the agent to remain useful without becoming unsafe.

4.5 State Update and Loop Trace

Each iteration should update the agent's state and append a **loop trace**. The trace is a structured record of what happened during execution.

A loop trace may include:

Field	Description
Iteration number	Current loop count
Goal state	Current understanding of the goal
Plan	Selected or revised plan
Action	Tool call, message, file edit, API request, or reasoning step
Observation	Result returned by the action
Evaluation	Progress, safety, confidence, and drift assessment
Controller decision	Continue, revise, rollback, escalate, or stop
Risk/cost state	Current budget and risk status
Reflective state	Lessons learned from the iteration

The loop trace is important for debugging and auditability. It allows developers and users to reconstruct why the agent took a particular action. It also supports accountability when agents operate in business, research, or production environments.

A useful loop trace should be structured enough for automated analysis but concise enough for human review. Overly verbose traces can create noise, privacy risks, and storage costs. Therefore, loop traces should support summarization, redaction, and selective retention.

4.6 Rollback and Recovery

Loop engineering requires recovery mechanisms. Agents will make mistakes, tools will fail, and observations will sometimes contradict expectations. A robust loop-engineered system should be able to recover without starting over.

Recovery mechanisms may include:

- Retrying a failed tool call.
- Revising the plan after an unexpected observation.
- Rolling back file changes.

4.6 Rollback and Recovery

Loop engineering requires recovery mechanisms. Agents will make mistakes, tools will fail, and observations will sometimes contradict expectations. A robust loop-engineered system should be able to recover without starting over.

Recovery mechanisms may include:

- Retrying a failed tool call.
- Revising the plan after an unexpected observation.
- Rolling back file changes.
- Asking the user to clarify ambiguous instructions.
- Switching tools when one source fails.
- Escalating to human review after repeated failures.

Rollback is especially important for agents that modify external environments. If an agent edits code, changes a database record, or updates a file, the system should preserve enough state to restore the previous condition if verification fails.

A practical rollback policy requires checkpoints. For example, before modifying a file, the system may store a snapshot or versioned copy. Before calling a write API, it may log the intended change and the prior state. If the post-action evaluator determines that the change is incorrect, the controller can restore the previous state or mark the task for human review.

A simple recovery policy may be expressed as:

```
If action fails:
    classify failure
    If failure is temporary:
        retry with limit
    Else if failure changes state:
```

```
        rollback if possible
    Else if failure affects plan:
        revise plan
    Else if failure repeats:
        escalate to human
```

This makes recovery a designed part of the loop rather than an afterthought.

4.7 Summary of the Framework

The proposed framework can be summarized as follows:

Component	Primary role	Failure it mitigates
Goal representation	Defines the task, constraints, success criteria, and stop conditions	Goal drift
State model	Maintains task memory, outputs, errors, and reflective lessons	Repetition, context loss
Action executor	Performs controlled tool use and external operations	Unsafe or unlogged actions
Observation collector	Captures actual tool results and errors	Confusing intention with outcome
Evaluator	Assesses progress, quality, confidence, and drift	Premature or endless continuation
Controller	Decides whether to continue, revise, rollback, escalate, or stop	Infinite loops, runaway cost, unsafe execution

The framework's central claim is that reliable AI agents require more than a capable language model. They require an execution architecture that can observe, remember, verify, govern, and stop.

---

5. Loop Types

Loop engineering does not require a single universal loop. Different tasks may require different loop structures. A loop-engineered agent may use one loop type or combine several depending on the task.

5.1 Planning Loop

The **planning loop** updates the agent's plan as new information appears. It is useful when the task is uncertain, open-ended, or dependent on external evidence.

Example:

`Initial plan → new evidence → revised plan → updated task sequence`

A planning loop is useful for research, software architecture, business strategy, and multi-step analysis.

5.2 Execution Loop

The **execution loop** performs concrete actions. It is useful for file creation, code generation, API calls, data extraction, and report production.

Example:

```
Write draft → run test → observe error → revise code → rerun test
```

The execution loop should be bounded by iteration limits, cost budgets, and safety checks.

5.3 Verification Loop

The **verification loop** checks whether outputs meet requirements. It may use tests, rubrics, reviewers, or deterministic validation.

Example:

```
Generate output → check against rubric → identify gaps → revise output
```

Verification loops are especially important for code, research papers, reports, and decision-support outputs.

5.4 Reflection Loop

The **reflection loop** analyzes failures and stores lessons for future use.

Example:

```
Attempt failed → diagnose failure → store lesson → adjust future plan
```

Reflection loops are inspired by Reflexion, but loop engineering treats reflection as an operational state component rather than only a model prompt (Shinn et al., 2023).

5.5 Governance Loop

The **governance loop** monitors risk, permissions, and human oversight.

Example:

```
Proposed action → risk classification → approval decision → execution or rejection
```

Governance loops are essential when agents can modify files, spend money, send messages, or interact with external systems.

---

6. Qualitative Evaluation Framework

Because this paper is conceptual, it does not claim definitive empirical proof. Instead, it proposes a qualitative evaluation framework that can guide future experiments, case studies, or benchmarks.

6.1 Evaluation Dimensions

Dimension	Evaluation question	Strong evidence	Weak evidence
Goal fidelity	Does the agent remain aligned with the original task?	Requirements preserved across iterations	Output drifts from original goal
State continuity	Does the agent preserve requirements, outputs, and errors?	Prior actions and observations inform next steps	Agent repeats or forgets earlier work
Recovery	Can the agent recover after errors?	Revises plan after tool failure	Stops or loops after first error
Termination	Does the agent stop appropriately?	Stops when goal is achieved or progress stalls	Continues unnecessarily or stops prematurely
Governance	Does the agent avoid unsafe or unauthorized actions?	High-risk actions require approval	Agent acts without permission
Traceability	Are actions and decisions logged?	Loop trace supports audit	Decisions are opaque
Cost awareness	Does the agent manage resources?	Stops when marginal value declines	Continues despite high cost
Human escalation	Does the agent ask for help when needed?	Escalates ambiguous or high-risk decisions	Proceeds despite uncertainty

6.2 Expected Comparative Behavior

Rather than presenting unsupported percentages, this paper offers expected qualitative behavior across three common agent designs.

Agent design	Description	Expected strengths	Expected weaknesses
One-shot agent	Receives task and produces final output without tool use or revision	Fast, simple, low cost	High goal drift, no recovery, weak verification
continue			

Ethics Statement

No human subjects, animal subjects, private datasets, or regulated decision-making systems were used in this paper. The paper is conceptual and does not report empirical studies involving human participants. Future empirical work on loop-engineered agents should follow appropriate ethics review procedures, especially when agents interact with users, process sensitive data, or influence real-world decisions.

Data Availability

No original dataset was generated for this paper. The paper is a conceptual framework paper and includes a proposed evaluation rubric rather than empirical results.

Funding

The author received no specific grant from any funding agency for this paper.

Conflict of Interest

The author declares no conflicts of interest.

Author Contribution

Mike Oller, AI Tool Insider, is the sole author of this paper.

References

Boyd, J. R. (1987). *A discourse on winning and losing*. Air University.

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly, 28*(1), 75–105. https://doi.org/10.2307/25148625

LangChain. (2024). *LangGraph: Building stateful, multi-actor applications with LLMs*. https://langchain-ai.github.io/langgraph/

Russell, S., & Norvig, P. (2021). *Artificial intelligence: A modern approach* (4th ed.). Pearson.

Shinn, N., Labash, B., & Gopinath, A. (2023). *Reflexion: An autonomous agent with dynamic memory and self-reflection*. arXiv. https://arxiv.org/abs/2303.11366

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press.

Wiener, N. (1948). *Cybernetics: Or control and communication in the animal and the machine*. MIT Press.

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A., White, R. W., Burger, D., & Wang, C. (2023). *AutoGen: Enabling next-gen LLM applications via multi-agent conversation*. arXiv. https://arxiv.org/abs/2308.08155

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. *Proceedings of the International Conference on Learning Representations*. https://arxiv.org/abs/2210.03629

Appendix A: Minimal Controller Pseudocode

```
Initialize:
    state = create_state(goal)
    trace = []
    iteration = 0

While iteration < max_iterations:
    state = update_state(state)

    risk_level = assess_risk(state)
    cost_level = assess_cost(state)

    If risk_level == "critical":
        decision = "escalate_to_human"
    Else If confidence(state) < low_confidence_threshold:
        decision = "gather_more_evidence"
    Else If progress_score(state) < minimum_progress_threshold:
        decision = "revise_plan"
    Else If goal_achieved(state):
        decision = "terminate_success"
    Else If entropy(state) > entropy_threshold:
        decision = "clarify_or_escalate"
    Else If cost_level >= cost_budget:
        decision = "terminate_or_escalate"
    Else:
        decision = "execute_next_action"

    append trace(state, decision)

    If decision == "terminate_success":
        break
    Else If decision == "terminate_or_escalate":
        request_human_review()
```

```
        break
    Else If decision == "escalate_to_human":
        request_human_review()
        break
    Else If decision == "revise_plan":
        revise_plan(state)
    Else If decision == "gather_more_evidence":
        gather_evidence(state)
    Else:
        execute_action(state)

    iteration += 1

If iteration >= max_iterations:
    decision = "terminate_with_warning_or_escalate"
```

Appendix B: Suggested Loop Trace Schema

```
{
  "iteration": 1,
  "goal": {
    "task": "string",
    "success_criteria": ["string"],
    "constraints": ["string"]
  },
  "plan": {
    "current_step": "string",
    "next_action": "string"
  },
  "action": {
    "type": "tool_call | file_edit | api_call | message | reasoning_step",
    "target": "string",
    "input_summary": "string"
  },
```

```
  "observation": {
    "status": "success | failure | partial",
    "output_summary": "string",
    "error": "string or null"
  },
  "evaluation": {
    "confidence": "low | medium | high",
    "progress": "low | medium | high | negative",
    "goal_drift": "low | medium | high",
    "risk": "low | medium | high | critical"
  },
  "controller_decision": "continue | revise | rollback | escalate | stop",
  "reflective_state": {
    "lessons_learned": ["string"]
  }
}
```

Appendix C: Recommended Reporting Format for Future Empirical Studies

Future studies evaluating loop-engineered agents should report both **outcome metrics** and **process metrics**.

Outcome Metrics

Metric	Description
Task success	Whether the final output satisfies the task goal
Output quality	Human or automated evaluation of final artifact quality
Correctness	Factual, logical, or technical accuracy
Completeness	Whether all required components are present

Process Metrics

Metric	Description
Iteration count	Number of loop cycles required
Recovery rate	Whether the agent recovers after errors

Metric	Description
Goal drift	Degree of deviation from the original task
Governance compliance	Whether unsafe or unauthorized actions were blocked
Trace completeness	Whether actions, observations, and decisions were logged
Cost	Token, API, compute, and human-review cost
Latency	Total execution time

Reporting both categories is important because an agent can produce a good final result while still behaving inefficiently or unsafely during execution.

Appendix D: Compact Framework Summary

```
Loop Engineering Framework


1. Goal Representation
   - Define task, constraints, success criteria, and stop conditions.


2. State Model
   - Preserve static, dynamic, tool, reflective, and governance state.


3. Action Executor
   - Perform controlled tool use, file edits, API calls, or messages.


4. Observation Collector
   - Capture actual outputs, errors, and environmental changes.


5. Evaluator
   - Assess progress, confidence, quality, drift, and safety.


6. Controller
   - Decide whether to continue, revise, rollback, escalate, or stop.


7. Loop Trace
   - Log actions, observations, evaluations, and decisions.
```

8. Governance Policies

- Enforce risk budgets, cost budgets, confidence thresholds, and human escalation.